```
    {                                   ❷
    }                                   ❷
}
```

❶ The LogResourceFilter is added to the Razor Page's PageModel.

❷ The filter applies to every page handler in the page.

Filters you apply as attributes to controllers, actions, and Razor Pages are automatically discovered by the framework when your application starts up. For common attributes, you can go one step further and apply filters globally without having to decorate individual classes.

You add global filters in a different way from controller- or action-scoped filters—by adding a filter directly to the MVC services when configuring your controllers and Razor Pages. The next listing shows three equivalent ways to add a globally scoped filter.

Listing 21.6 Applying filters globally to an application

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers(options =>              ❶
{
    options.Filters.Add(new LogResourceFilter());       ❷
    options.Filters.Add(typeof(LogResourceFilter));     ❸
    options.Filters.Add<LogResourceFilter>();           ❹
});
```

❶ Adds filters using the MvcOptions object

❷ You can pass an instance of the filter directly. . .

❸ . . . or pass in the Type of the filter and let the framework create it.

❹ Alternatively, the framework can create a global filter using a generic type parameter.

You can configure the `MvcOptions` by using the `AddControllers()` overload. When you configure filters globally, they apply both to controllers *and* to any Razor Pages in your application. If you wish to configure a global filter for a Razor Pages application, there